

Relatório de Aprendizagem

Sistema RAG Distribuído via Tailscale — Tema 5

Aluno: Pablo Abdon **Disciplina:** Engenharia de Contexto e Harness Engineering aplicada à Programação Distribuída e Paralela **Instituição:** CESUPA — 2º bimestre de 2026 **Período relatado:** fechamento dos Blocos B3, B4 e B5

Sumário

1. Introdução
2. Contribuição técnica
3. Aprendizados técnicos 3.1 Workers de fila e o gap de observabilidade 3.2 Scripts operacionais e o ciclo de vida das mensagens 3.3 Ambiente local heterogêneo: GPU AMD num projeto pensado para NVIDIA
4. Aprendizados de processo
5. Conclusão

1. Introdução

Este relatório documenta o que aprendi ao integrar o projeto do Tema 5 da disciplina de Programação Distribuída e Paralela no fechamento operacional dos Blocos B3, B4 e B5. Entrei quando o sistema já tinha pipeline, filas, prompts, cache, observabilidade parcial e documentação técnica — o que faltava era destravar pontos práticos para validar a demo e o dashboard ponta-a-ponta. Procuro registrar tanto os ganhos técnicos — expor `/metrics` em workers de fila, construir scripts operacionais para corpus e DLQs, e lidar com um ambiente sem GPU NVIDIA — quanto os de processo, em especial sobre a diferença entre fazer um sistema *funcionar* e conseguir *operá-lo*.

2. Contribuição técnica

Fui responsável pelo fechamento operacional do projeto: implementei o servidor de métricas dos workers em `src/shared/workers_metrics_server.py` usando `aiohttp`, reaproveitando o registro Prometheus já existente em `src/shared/metrics.py`; integrei o servidor no bootstrap de `src/workers/ingest/main.py` e `src/workers/query/main.py`; expus as portas 9100/9101/9102 no `docker-compose.yml` para os três workers; adicionei a configuração `worker_metrics_port` em `src/shared/config.py` e os testes unitários em `tests/unit/test_workers_metrics_server.py`. Criei também os scripts operacionais — `scripts/seed_corpus.py` (envio de corpus para POST `/ingest`), `scripts/dlq_inspector.py` (list/replay/purge das DLQs) e `scripts/chaos_test.sh/chaos_test.ps1` (cenários de falha controlada) — além do corpus mínimo em `samples/corpus/` e do conjunto de avaliação `data/eval_queries.jsonl` com 30 queries em PT/EN. Por fim, atualizei a documentação (`docs/decisoes.md`, `docs/prompts.md`, `slides`, `ENTREGA.md`, `README.md`, `TODO.md`) e validei a stack localmente, incluindo a criação de um `docker-compose.override.yml` para destravar o Ollama num ambiente com GPU AMD.

3. Aprendizados técnicos

3.1 Workers de fila e o gap de observabilidade

- **Workers não são serviços HTTP:** consomem trabalho via mensageria, não via requisição do usuário — o Prometheus, que faz *scrape* por HTTP, não tinha onde se conectar. O job `workers` ficava DOWN e parte dos painéis do Grafana ficava sem dados, mesmo com o pipeline funcionando corretamente.
- **Servidor HTTP mínimo embutido:** a solução foi subir um pequeno `aiohttp` dentro de cada worker, expondo `/metrics` em portas distintas. Não foi preciso inventar métricas novas — bas-

tou reaproveitar o registro Prometheus que já vivia em `src/shared/metrics.py`. O trabalho real era expor o que já existia no processo certo.

- **Métricas como contrato de operação:** depois de expor, validei que `http://localhost:9100/metrics`, `9101` e `9102` retornavam métricas `rag_*` e que o Prometheus passou a marcar os três workers como UP. Esse fechamento simples é o que viabilizou alertas reais sobre o estado dos consumidores.
- **Citation com página "null" como string:** durante a validação final, apareceu um caso em que o worker recebeu a página da citação como texto "null" e o schema esperava int. A mensagem foi reprocessada com sucesso, mas ficou anotado como ponto de melhoria — normalizar `null/"null"/"n/a"` antes de montar a citação final.

3.2 Scripts operacionais e o ciclo de vida das mensagens

- **Seed de corpus repetível:** `scripts/seed_corpus.py` lê PDFs, Markdown e HTML de uma pasta, converte para base64 e envia para `POST /ingest`, registrando quantos foram aceitos e quantos falharam. Popular o Qdrant de forma reproduzível é o que permite repetir experimentos sem depender de upload manual a cada rodada.
- **DLQ inspector com três verbos:** `list`, `replay --limit N` e `purge --yes` cobrem o ciclo completo de mensagens que falharam — inspecionar, reprocessar com cuidado, e descartar quando não tem mais sentido reprocessar. Sem essas três operações, uma mensagem na DLQ era um beco sem saída.
- **Chaos test como evidência, não como teste:** os scripts `.sh/.ps1` param e religam o worker de chunks, param e religam o Ollama, e disparam uma rajada de queries concorrentes. Não substituem testes unitários — geram evidência de que o sistema sobrevive a falhas reais, e o resultado vira input para o doc técnico (B3, Exp 4).
- **Corpus mínimo versionável:** `samples/corpus/` tem documentos Markdown originais sobre engenharia de software, RAG, arquitetura distribuída, observabilidade e padrões de projeto. Não substitui um corpus grande, mas permite validar o funcionamento local sem commitar arquivos pesados ou de terceiros.

3.3 Ambiente local heterogêneo: GPU AMD num projeto pensado para NVIDIA

- **Falha de container ≠ bug da aplicação:** o `docker-compose.yml` reservava GPU NVIDIA para o Ollama. Numa máquina com GPU AMD isso quebrava no `docker compose up` antes do código rodar. O sinal de erro vinha do Docker, não da aplicação — isso ensinou a distinguir as duas camadas antes de procurar bug no lugar errado.
- **Override local ignorado pelo Git:** a solução foi criar `docker-compose.override.yml` removendo a reserva de GPU NVIDIA, intencionalmente ignorado pelo Git por ser específico do ambiente. O Compose faz merge automático com o arquivo principal, então o `up` voltou a funcionar sem alterar o arquivo versionado.
- **Modelo carregando em CPU:** na primeira execução do smoke após a stack subir, o tempo de resposta foi alto — não por bug, mas porque o Ollama estava carregando `llama3.2:1b` em CPU pura. Na segunda execução, com o modelo já em memória, o smoke fechou normalmente com `[smoke] OK`. Em IA local sem GPU, latência alta nem sempre é defeito; às vezes é só o custo do hardware disponível.
- **Modelos leves para CPU vs modelo cheio para GPU:** o projeto usa `llama3.2:1b` (geração) e `omic-embed-text` (embeddings) quando GPU NVIDIA não está disponível. No PC1 com GPU, o modelo de geração vira `qwen2.5:7b-instruct`. A escolha é parametrizada — não há código diferente por hardware, só configuração.

4. Aprendizados de processo

- **Operar é diferente de implementar:** entrei com o pipeline funcionando, e o trabalho mais valioso não foi escrever feature nova mas destravar pontos que impediam operação real — `/metrics` em workers, scripts para popular corpus, DLQ inspector, override de GPU. Em sistema

distribuído, fazer funcionar não basta; é preciso conseguir ver, testar, popular, e inspecionar — e essa segunda camada é tão construída quanto a primeira.

- **Tailscale como infraestrutura habilitadora:** não é parte da lógica de RAG, mas é o que permite transformar PCs separados em uma rede privada onde os workers acessam os serviços centrais com segurança. Esse tipo de componente fica “invisível” quando funciona, mas determina se o sistema distribuído realmente existe ou é só um plano no papel.
- **Evidência operacional como entregável:** as validações de stack — Prometheus mostrando workers UP, Grafana com dashboard provisionado, Loki ready, Qdrant com points_count=6, smoke fechando com [smoke] OK — ficaram registradas em data/evidencias-operacionais-pablo.md. Não substituem a validação distribuída com o PC1, mas ancoram a parte local em fatos verificáveis em vez de “funcionou aqui”.
- **Documentação técnica como ferramenta de coordenação:** atualizar decisoes.md, prompts.md, slides, ENTREGA.md, README.md, TODO.md e PENDENCIAS_FINAIS.md no fechamento não foi burocracia — foi como o time mantém o estado real do projeto sincronizado entre quem entra e sai das sessões. Documentação aqui precede o código por design, e seguiu essa convenção.

5. Conclusão

A minha entrada no projeto foi no fechamento operacional, e o que ficou como aprendizado mais duradouro é a distinção entre fazer um sistema *funcionar* e conseguir *operá-lo*. O /metrics nos workers foi um caso pequeno mas exemplar: o pipeline já funcionava, mas o sistema era invisível para o Prometheus até esse endpoint existir. Os scripts de seed, DLQ inspector e chaos test pertencem à mesma família — não são funcionalidades para o usuário final, são alavancas para quem opera o sistema reagir a falhas, popular dados e medir comportamento. Em paralelo, a experiência com GPU AMD num projeto pensado para NVIDIA consolidou uma lição que eu sabia “no abstrato” mas confundia na prática: nem toda falha de container é bug da aplicação. Os próximos passos são de ambiente — confirmar o IP da tailnet, validar conectividade com o PC1, subir esta máquina como worker no modo distribuído. A partir daí, minha parte deixa de ser só “escrever código” e passa a garantir que esta máquina realmente participa do sistema.